Week 2 Worked Companion

Using the worked companion

Attempt the student exercises independently first. This companion contains hints and complete solutions. Read only as far as needed, then return to your attempt.

Examples are original teaching examples. Follow each official link for the source problem and its complete contract.

Student exercises

Hints and solutions

Instructor guide

Download Week 2 code

3 · Staged hints

LeetCode 3: Longest Substring Without Repeating Characters · Medium

New exercise

H1. Are all characters in the current interval distinct?

H2. How many left moves are needed to remove a duplicate?

H3. Remove leftmost characters until the incoming character is absent, then add it.

3 · Correctness and cost

After insertion, the set represents the unique-character window. Its left boundary is the earliest feasible one for that right endpoint. Each pointer advances at most n times.

n is length, a alphabet size. Expected O(n) time, O(min(n,a)) space.

3 · Counterexamples and checks

A single removal on “abba” leaves b duplicated. Empty text gives 0; one space gives 1.

```
text="abba c" → 4 ("ba c")
```

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

3 · Complete implementation

H4. Compare the complete implementation with your state trace.

```
def longest_unique(text):
    active, left, best = set(), 0, 0
    for right, char in enumerate(text):
        while char in active:
            active.remove(text[left])
            left += 1
        active.add(char)
        best = max(best, right - left + 1)
    return best
```

209 · Staged hints

LeetCode 209: Minimum Size Subarray Sum · Medium

New exercise

H1. Must every interval be summed again?

H2. How does removing the left value change the sum?

H3. Extend right, then record and shrink while the sum qualifies.

209 · Correctness and cost

Positivity makes shrinking decrease the sum. Record each valid window before shrinking. A discarded left boundary can only produce longer later windows.

n is length. O(n) total pointer movements and time; O(1) auxiliary space.

209 · Counterexamples and checks

Outside the positive-only contract, target=3, [1,-1,3] can make this rule miss length 1.

`target=6, nums=[1,2,4,2] → 2`

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

209 · Complete implementation

H4. Compare the complete implementation with your state trace.

```
def min_subarray(target, nums):
    total = left = 0
    best = len(nums) + 1
    for right, value in enumerate(nums):
        total += value
        while total >= target:
            best = min(best, right - left + 1)
            total -= nums[left]
            left += 1
    return 0 if best > len(nums) else best
```

155 · Staged hints

LeetCode 155: Min Stack · Medium

New exercise

H1. How can the previous minimum be restored after a pop?

H2. What happens when the minimum occurs twice?

H3. Store (value, minimum through this depth) per entry. Create entries separately for each instance.

155 · Correctness and cost

Induction on depth gives the correct prefix minimum; popping the last pair restores the previous prefix minimum.

d is depth. Queries O(1), Python list push/pop amortized O(1), space O(d). Separate abstract constant-time operations from resizing.

155 · Counterexamples and checks

Calling min over the stack costs O(d). A single current minimum loses its predecessor after a pop.

`push(2),push(2),push(5),pop(),pop(),getMin() → 2`

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

155 · Complete implementation

H4. Compare the complete implementation with your state trace.

```
class MinStack:
    def __init__(self):
        self.entries = []

    def push(self, val):
        minimum = min(val, self.entries[-1][1]) if self.entries else val
        self.entries.append((val, minimum))

    def pop(self):
        self.entries.pop()

    def top(self):
        return self.entries[-1][0]

    def getMin(self):
        return self.entries[-1][1]
```

150 · Staged hints

LeetCode 150: Evaluate Reverse Polish Notation · Medium

New exercise

H1. Is the first popped value the left or right operand?

H2. Which way should -5/2 be truncated?

H3. Push numbers; pop right then left for an operator and push its result. Divide absolute integers and restore the sign.

150 · Correctness and cost

The stack preserves completed subexpression values in order. Each operator combines the final two; a valid complete expression leaves one value.

t is token count. O(t) time and O(t) space under the bounded-integer model.

150 · Counterexamples and checks

5//-2 is -3, but truncation toward zero gives -2. Reversing operands changes subtraction.

`tokens=["9","4","-","-2","/"] → -2`

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

150 · Complete implementation

H4. Compare the complete implementation with your state trace.

```
def rpn(tokens):
    stack = []
    for token in tokens:
        if token not in {"+", "-", "*", "/"}:
            stack.append(int(token))
            continue
        right, left = stack.pop(), stack.pop()
        if token == "+":
            value = left + right
        elif token == "-":
            value = left - right
        elif token == "*":
            value = left * right
        else:
            quotient = abs(left) // abs(right)
            value = -quotient if (left < 0) != (right < 0) else quotient
        stack.append(value)
    return stack[0]
```

Session 3 answer checks

1. The last fixed-width sum is 6. At right=3, remove index 1 and add index 3.
2. On the second b in “abba”, removing only a leaves the earlier b. A set cannot represent the repeated b in that interval.
3. Both pointers move only forward, at most n positions each. No iteration resets the left pointer.
4. The positive-only rule returns 3; the true shortest length is 1. Removing -1 would increase the sum, so the stopping argument fails.

Session 4 answer checks

1. The minimum is 2; the first (2,2) pair remains after the second is removed.
2. Push 2 into one instance and 7 into another. Their top values must be 2 and 7, and the second minimum must be 7. Shared state fails these checks.
3. The first pop returns the right operand, 4. The subtraction is 9-4.
4. Python floor division gives -3; truncation toward zero gives -2.
5. Scanning all d entries costs O(d), replacing a direct O(1) lookup.

Execution and sources

```
cd code
python demo_03.py
python demo_04.py
python verify.py
```

Completed implementations are in `solutions.py`; independent work is in `student_week_02.py`. On LeetCode, adapt each function to the official method signature.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).